Orquestração de agentes no NarraTer:
comparação de quatro versões numa mesma tarefa de programação

Gabriel Cortez Claude Code

15 de outubro de 2026

Resumo

Rodamos a mesma tarefa de programação em quatro versões do NarraTer, de julho a outubro de 2026, três vezes cada, sempre com o mesmo ponto de partida e os mesmos modelos: um Narrador (Claude Opus 5.5) coordenando dois desenvolvedores (Claude Sonnet 5.5 e Claude Haiku 5.5). As doze rodadas entregaram tudo o que foi pedido: 31 de 31 testes escondidos. A tarefa, portanto, não separa as versões pela qualidade. As versões se separam no custo. A mais antiga (A, modelo cooperativo com MCP) foi a mais rápida e a mais barata: 2,2 min e 1,8 milhão de tokens processados em média. A atual (D) gastou 4,6 min e 3,4 milhões. A latência do protocolo em si é de segundos e não explica a diferença, que vem do número de rodadas de trabalho que o Narrador encomenda. Um mecanismo concreto aparece em todas as rodadas de C e D e em duas de B: a resposta de um `ask`, lida da tela do agente, chega cortada, e o Narrador precisa pedir de novo. Na versão A, em que o agente envia o relatório inteiro por mensagem, isso não aconteceu nenhuma vez. Com três rodadas por versão, nenhuma diferença passa de $p = 0{,}10$ (o menor valor possível nesse tamanho de amostra). Os resultados são indicações fortes, não provas.

1 Objetivo

O NarraTer é um canvas de terminais em que agentes de IA conversam por arestas. Entre julho e outubro de 2026 o jeito de orquestrar mudou três vezes: saímos de um modelo cooperativo preso ao Claude Code (MCP, *prompt* de sistema extra, *hook* de parada) para um protocolo que trata o terminal como API universal (decisão D01), depois tornamos o `ask` assíncrono e durável (D07, D08) e, por fim, refizemos o leitor de estado que decide quando um agente terminou (D10–D12). A pergunta deste estudo é o que cada mudança custou ou rendeu em quatro eixos: **tokens**, **contexto**, **eficiência** (tempo e coordenação) e **trabalho feito**.

2 Versões comparadas

	Commit	Data	O que muda na orquestração
A	`e708d7b`	13/07	Modelo cooperativo. Servidor MCP `narrater-mcp` (Python), *prompt* via `-append-system-prompt`, *hook* `Stop` avisa que o agente parou. Arestas dirigidas. O agente responde com `reply_message`/`send_message`: o texto inteiro chega ao outro lado. O papel de líder proíbe Bash, Edit e Write. Só funciona com Claude Code.
B	`16cee0e`	12/10	Protocolo agnóstico (D01). CLI `narrater` em Rust ensinada por *skill*; papéis como `CLAUDE.md` em `.narrater/roles/`; sessões em tmux. O `ask` bloqueia até o fim do turno do alvo, lido da tela, e devolve o texto da tela. O líder não pode editar arquivos.
C	`95fb11a`	14/10	`ask` assíncrono (`-no-wait`), `narrater wait` e pedidos duráveis em `asks.jsonl` (D07, D08).
D	`db3d535`	15/10	Leitor de estado novo (título da janela, primeiro plano do tmux, permissão; D10–D12), *statusline* de uso (D17) e o rework visual. *Skills* e papéis idênticos aos de C.

Tabela 1: As quatro versões. C e D diferem só no backend de leitura de estado, na *statusline* e na interface.

3 Método

3.1 Tarefa

Um repositório Python inventado (`pedidos`, cerca de 300 linhas, só biblioteca padrão) com regras de negócio próprias em `REGRAS.md`: desconto progressivo por faixas, imposto proporcional, frete. O Narrador recebe um único pedido:

1. implementar `docs/PEDIDO-MULTIMOEDA.md`: pedidos em BRL, USD ou EUR, tabela de câmbio com a taxa da data anterior mais recente, regras aplicadas em reais e novos campos no relatório. Atravessa os três módulos e exige combinar uma interface;
2. corrigir duas reclamações de `docs/RECLAMACOES.md`, sem saber onde está o erro: o desconto da terceira faixa calculado sobre a base errada, e linhas de um mesmo pedido intercaladas no CSV virando dois pedidos (`groupby` sem ordenação);
3. manter os testes existentes passando e escrever testes novos.

Preferimos isso a um exercício clássico (uma calculadora de expressões) porque o modelo saberia o exercício clássico de cor: o líder o faria sozinho e todas as versões tirariam nota máxima. O texto do pedido é o mesmo em todas as rodadas e termina com "escreva `TAREFA CONCLUIDA`".

3.2 Julgamento

31 testes escondidos, que os agentes nunca veem, divididos em quatro níveis: *Básico* (7), *Bordas* (12), *Defeitos* (5) e *Regressão* (7). Os valores esperados foram calculados à mão a partir das regras. Uma solução de referência passa 31/31, e o repositório inicial passa 5/31 (só regressão).

3.3 Condições

- **Ponto de partida zerado em toda rodada**: `HOME` próprio (configuração do Claude, *skills*, sessões, memória, `~/.config/narrater`), servidor tmux próprio, pasta de trabalho copiada do zero, ambiente mínimo (só login e proxy) e sessões novas (`-session-id` gerado na hora).
- **Modelos fixos por papel**: Narrador = `claude-opus-5-5`, DevA = `claude-sonnet-5-5`, DevB = `claude-haiku-5-5`, Claude Code 2.1.300. Um *wrapper* `claude` na frente do `PATH` faz isso igual em todas as versões. O mesmo *wrapper* entrega o pedido ao Narrador como *prompt* inicial, 25 s depois de ele subir, sem digitar na interface.
- **Papéis**: os papéis padrão de cada versão (*leader* e *coder*), com o texto que o seletor de agentes copia. Permissões ignoradas (`-dangerously-skip-permissions`, com `IS_SANDBOX=1`, porque o contêiner roda como root).
- **Ordem contrabalançada**: A1 B1 C1 D1, D2 C2 B2 A2, C3 A3 D3 B3. O *cache* de *prompt* do servidor dura até 1 h e baratearia quem roda logo depois de outra versão.
- **Fim**: o Narrador escreve a linha de fim, ou 40 minutos (nenhuma rodada chegou perto).

3.4 Métricas

Tudo vem das transcrições do Claude Code (`~/.claude/projects/*/<sessão>.jsonl`), que têm o mesmo formato nas quatro versões.

- **Tokens processados**: entrada + *cache* lido + *cache* criado, somados por chamada à API (cada `message.id` contado uma vez). É o tamanho do contexto que o modelo leu e não muda se o *cache* acertou ou não. O custo em US$ é o preço de API equivalente (Opus 5.5: 4/20; Sonnet 5.5: 2/10; Haiku 5.5: 0,10/0,50 por milhão de tokens, *cache* lido a 0,1× e criado em 1 h a 2× a entrada). A conta usada é de assinatura e não paga por token.

- **Contexto**: contexto da primeira chamada (o custo fixo do agente antes da tarefa), pico e compactações.
- **Tempo**: do pedido entregue ao Narrador até a linha de fim. Um dev está "trabalhando" entre receber uma fala e encerrar o turno; o resto é tempo só do Narrador.
- **Latências do protocolo**: *entrega* (o líder pede e a fala aparece no dev); *detecção* (o dev termina o turno e chega a resposta ao líder, que já esperava; num `ask -batch` ou `wait` sem ids só conta o último a terminar); *volta* (versão A: o dev envia e a fala aparece no líder).
- **Coordenação**: pedidos do Narrador aos devs e, entre eles, os **repetidos porque a resposta veio cortada**. Esses foram detectados por palavras ("cortado", "repita", "relatório completo"...) e conferidos à mão.

4 Resultados

4.1 Trabalho feito

	A	B	C	D
Concluiu (fração)	1.00 (1.00–1.00)	1.00 (1.00–1.00)	1.00 (1.00–1.00)	1.00 (1.00–1.00)
Duração (min)	2.2 (2.2–2.2)	2.9 (2.4–3.7)	3.5 (2.8–4.2)	4.6 (3.9–5.6)
Testes escondidos (de 31)	31.0 (31.0–31.0)	31.0 (31.0–31.0)	31.0 (31.0–31.0)	31.0 (31.0–31.0)
Básico (7)	7.0 (7.0–7.0)	7.0 (7.0–7.0)	7.0 (7.0–7.0)	7.0 (7.0–7.0)
Bordas (12)	12.0 (12.0–12.0)	12.0 (12.0–12.0)	12.0 (12.0–12.0)	12.0 (12.0–12.0)
Defeitos (5)	5.0 (5.0–5.0)	5.0 (5.0–5.0)	5.0 (5.0–5.0)	5.0 (5.0–5.0)
Regressão (7)	7.0 (7.0–7.0)	7.0 (7.0–7.0)	7.0 (7.0–7.0)	7.0 (7.0–7.0)
Testes da equipe	49 (42–56)	50 (44–55)	49 (41–53)	50 (45–60)
Linhas no diff	390 (349–444)	394 (336–424)	389 (301–462)	397 (322–511)

Tabela 2: Média e (mínimo–máximo) das três rodadas de cada versão.

[medido] As doze rodadas terminaram sozinhas e passaram nos 31 testes escondidos. Os testes da própria equipe passaram em todas (de 41 a 60 testes). A tarefa não separa as versões pela qualidade: é o efeito teto que tentamos evitar trocando a calculadora pelo repositório inventado, e ele aconteceu mesmo assim. Em nenhuma rodada o Narrador editou um arquivo: o papel de líder foi respeitado nas quatro versões.

4.2 Tempo e tokens

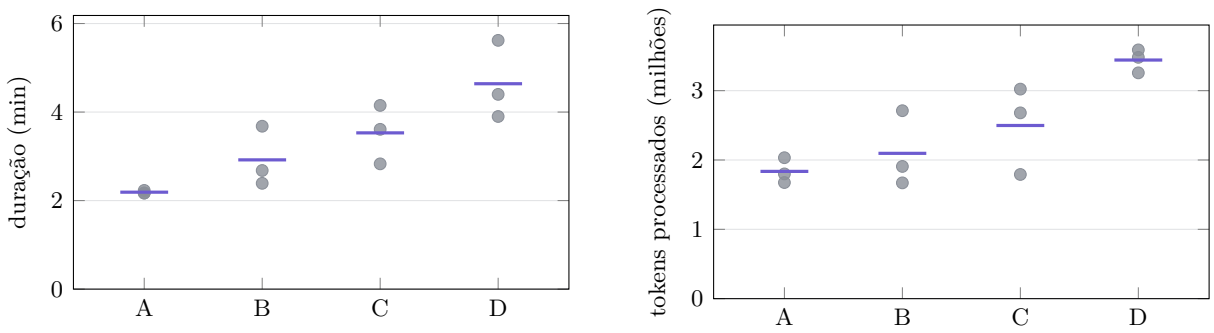


Figura 1: Cada ponto é uma rodada; o traço é a média da versão.

	A	B	C	D
Tokens processados (mil)	1836 (1675–2033)	2096 (1670–2711)	2498 (1791–3022)	3442 (3257–3588)
Narrador	592 (454–704)	562 (473–634)	682 (593–826)	740 (544–931)
DevA	475 (399–571)	501 (359–689)	730 (399–945)	1093 (935–1220)
DevB	768 (724–822)	1033 (581–1444)	1086 (800–1449)	1608 (1492–1718)
Tokens de saída (mil)	28.1 (27.0–29.2)	32.2 (27.1–39.7)	37.7 (29.1–45.8)	48.1 (44.3–55.8)
Custo de API equivalente (US$)	0.69 (0.61–0.77)	0.73 (0.68–0.77)	0.85 (0.65–0.99)	1.02 (0.97–1.09)
Chamadas à API	42 (37–47)	45 (39–55)	53 (42–60)	66 (63–69)

Tabela 3: Tokens e custo por rodada (média e mínimo–máximo).

[medido] A ordem se repete em tempo, tokens e custo: A < B < C < D. Nas três rodadas, a A foi mais rápida e processou menos tokens que qualquer rodada da D. A média da D é 2,1× mais longa e processa 1,9× mais tokens. O custo de API equivalente vai de US$ 0,69 a US$ 1,02 por tarefa. O Narrador (Opus) responde por cerca de metade do custo em todas as versões. O DevB (Haiku) é quem mais processa tokens e o que menos custa.

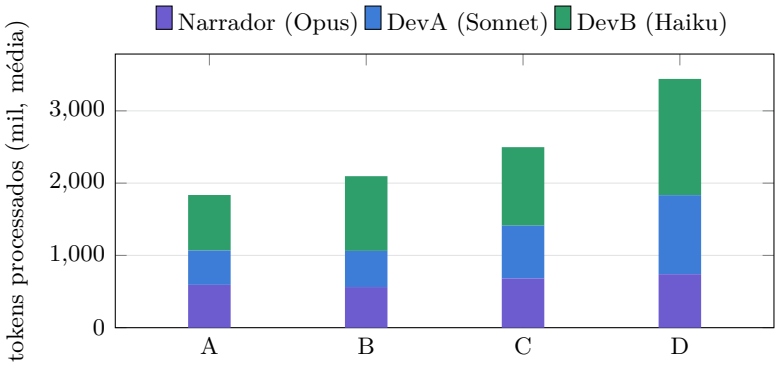


Figura 2: O que cresce de A para D são os tokens dos devs; o Narrador fica parecido.

4.3 Contexto

	A	B	C	D
Contexto inicial, Narrador (mil)	31.1 (31.1–31.1)	30.4 (30.4–30.4)	30.4 (30.4–30.4)	30.4 (30.4–30.4)
Contexto inicial, DevA (mil)	31.9 (31.8–32.1)	31.6 (31.6–31.7)	31.8 (31.7–31.9)	31.6 (31.6–31.6)
Contexto inicial, DevB (mil)	32.7 (32.6–32.9)	32.4 (32.4–32.5)	32.4 (32.4–32.4)	32.3 (32.3–32.4)
Pico de contexto, Narrador (mil)	48.8 (48.2–49.5)	47.5 (40.8–53.1)	45.5 (42.9–46.9)	45.0 (40.9–47.2)
Pico de contexto, DevA (mil)	45.2 (43.0–47.5)	50.6 (46.8–55.8)	56.2 (45.2–61.8)	65.2 (60.6–69.3)
Pico de contexto, DevB (mil)	60.3 (59.4–60.8)	68.3 (55.6–83.8)	72.9 (61.6–84.7)	85.5 (81.0–93.1)
Compactações	0.0 (0.0–0.0)	0.0 (0.0–0.0)	0.0 (0.0–0.0)	0.0 (0.0–0.0)

Tabela 4: Contexto em milhares de tokens.

[medido] O custo fixo de um agente, antes de qualquer trabalho, é de cerca de 30 a 33 mil tokens em todas as versões. Quase tudo é do próprio Claude Code (*prompt* de sistema e ferramentas). O protocolo do NarraTer pesa pouco. As ferramentas MCP e o *prompt* anexado da versão A custam ao Narrador cerca de 0,7 mil tokens a mais que a *skill* de B–D. Ninguém chegou perto do limite: o maior pico foi de 93 mil tokens (DevB na D1), e não houve nenhuma compactação. Os picos dos devs crescem de A para D junto com o número de turnos.

4.4 Para onde vai o tempo

	A	B	C	D
Mensagens entregues na tela de um agente	7.7 (5.0–9.0)	8.3 (5.0–10.0)	9.3 (5.0–12.0)	13.0 (11.0–16.0)
Chamadas de coordenação (CLI/MCP)	15.7 (12.0–20.0)	10.3 (7.0–16.0)	16.0 (10.0–20.0)	23.7 (21.0–28.0)
das quais consultas de estado	4.0 (3.0–6.0)	4.0 (2.0–6.0)	8.0 (6.0–9.0)	7.7 (7.0–8.0)
Chamadas à API do Narrador	14.3 (11.0–17.0)	14.3 (12.0–16.0)	17.7 (15.0–21.0)	19.3 (15.0–24.0)
Entrega (s)	0.3 (0.2–0.4)	0.4 (0.3–0.5)	0.4 (0.3–0.4)	0.4 (0.3–0.4)
Detecção (s)	–	2.2 (1.7–3.1)	1.8 (1.6–2.3)	1.7 (1.6–1.9)
Volta, só A (s)	0.3 (0.2–0.5)	–	–	–
Retorno até o líder saber (s)	50.8 (29.9–72.7)	17.2 (6.1–38.2)	4.5 (2.8–7.6)	3.0 (2.6–3.2)
Tempo com algum dev trabalhando (min)	1.6 (1.5–1.6)	1.6 (1.1–2.3)	2.3 (1.6–3.0)	3.6 (2.8–5.0)
Tempo só do Narrador (min)	0.6 (0.6–0.7)	1.3 (1.3–1.4)	1.2 (1.1–1.3)	1.0 (0.6–1.4)
Turnos de trabalho dos devs (soma)	4.3 (3.0–5.0)	7.0 (5.0–10.0)	8.0 (5.0–10.0)	11.0 (9.0–12.0)
Ocupação média dos devs	0.48 (0.45–0.49)	0.44 (0.41–0.50)	0.49 (0.36–0.59)	0.56 (0.51–0.63)
Fração com os dois devs ativos	0.23 (0.16–0.31)	0.35 (0.31–0.38)	0.32 (0.15–0.46)	0.37 (0.33–0.39)
Pedidos do Narrador aos devs	4.3 (3.0–5.0)	4.7 (3.0–6.0)	6.0 (4.0–8.0)	8.3 (7.0–10.0)
repetidos por resposta cortada	0.0 (0.0–0.0)	0.7 (0.0–1.0)	2.0 (1.0–3.0)	2.3 (1.0–5.0)
Narrador: chamadas de coordenação	10.7 (9.0–13.0)	10.0 (9.0–12.0)	13.7 (11.0–16.0)	16.3 (12.0–19.0)
Narrador: rodou os testes	0.0 (0.0–0.0)	0.0 (0.0–0.0)	0.0 (0.0–0.0)	0.0 (0.0–0.0)
Narrador: leituras de código	9.3 (9.0–10.0)	4.3 (0.0–9.0)	3.0 (0.0–8.0)	0.0 (0.0–0.0)
Narrador: edições (inclui via Bash)	0.0 (0.0–0.0)	0.0 (0.0–0.0)	0.0 (0.0–0.0)	0.0 (0.0–0.0)
DevA: chamadas Edit/Write	0.0 (0.0–0.0)	0.0 (0.0–0.0)	0.0 (0.0–0.0)	0.3 (0.0–1.0)
DevB: chamadas Edit/Write	7.0 (2.0–12.0)	6.3 (3.0–10.0)	6.3 (0.0–15.0)	10.7 (8.0–15.0)

Tabela 5: Coordenação. Latências: medianas por rodada, em segundos; entrega = pedido até a tela do dev; detecção = fim do turno até a resposta ao líder que esperava; volta = mensagem do dev até a tela do líder.

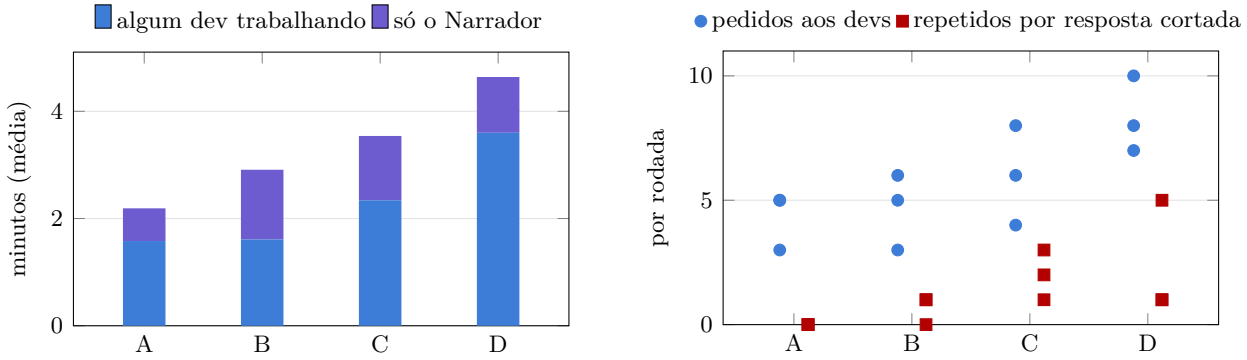


Figura 3: À esquerda, a duração decomposta; à direita, pedidos do Narrador por rodada.

[medido] A latência mecânica do protocolo é pequena perto da duração das rodadas, de dois a seis minutos:

- a *entrega* de um pedido leva de 0,2 a 0,5 s em todas as versões;
- a *volta* da versão A leva de 0,2 a 0,5 s;
- a *detecção* de fim de turno em B–D leva de 1,6 a 3,1 s. O leitor de estado de D não mudou isso nesta tarefa, e nenhum `ask` terminou em *timeout* ou em "precisa de você".

O que cresce de A para D é o trabalho encomendado:

- o Narrador faz 3 a 5 pedidos na A e 7 a 10 na D;
- os devs trabalham 3 a 5 turnos somados na A e 9 a 12 na D;
- o tempo com algum dev trabalhando vai de 1,6 para 3,6 min em média.

[medido] **Respostas cortadas.** Em B, C e D, a resposta de um `ask` é o texto que a tela do alvo mostra ao fim do turno: de 10 a 30 linhas. O tile de 380 px usado aqui tem 13 linhas visíveis nas versões B, C e D, medido numa cena de um só terminal. Quando o relatório do dev é maior, o Narrador recebe só o fim e pede de novo, com frases como "seu relatório ficou cortado na minha tela; envie completo com `narrater ask "Narrador"`" ou "repita só a última linha do unittest". As contagens foram:

- A: 0 em 3 rodadas;
- B: 1, 1 e 0;
- C: 2, 1 e 3;
- D: 1, 5 e 1.

Cada repetição custa um turno do dev e um do Narrador, com o contexto inteiro relido.

4.5 Testes estatísticos

	A×B	B×C	C×D	A×D
Duração	0 / 0.10	2 / 0.40	1 / 0.20	0 / 0.10
Tokens processados	4 / 1.00	3 / 0.70	0 / 0.10	0 / 0.10
Custo	3 / 0.70	3 / 0.70	1 / 0.20	0 / 0.10
Pedidos do Narrador	3.5 / 1.00	2.5 / 0.50	1.5 / 0.30	0 / 0.10
Turnos dos devs	1 / 0.30	4 / 1.00	1.5 / 0.30	0 / 0.10

Tabela 6: Mann–Whitney exato (U / p bilateral) entre pares de versões. Com três rodadas contra três, o menor p possível é 0,10, alcançado quando todas as rodadas de uma versão ficam acima de todas as da outra.

[medido] Entre A e D, todas as métricas da tabela atingem o extremo: em tempo, tokens, custo, pedidos e turnos, as rodadas de uma versão não se sobrepõem às da outra. Entre C e D, isso só vale para os tokens. Entre versões vizinhas, as diferenças de tempo se sobrepõem.

5 Discussão

1. **O custo da orquestração está nas rodadas de trabalho, não no transporte.** [medido] Entrega e detecção custam segundos, e o tempo total cresce em minutos. O que separa as versões é quantas vezes o Narrador volta aos devs (tabela 5). [inferido] Melhorar o leitor de estado (D10–D12) continua necessário para a correção do protocolo (nenhum *ask* errou aqui), mas não é onde está o ganho de eficiência.

2. **Puxar a resposta da tela custa caro; empurrar o relatório não.** [medido] A única diferença de mecanismo que aparece rodada a rodada é a resposta cortada: zero na A, de 1 a 5 por rodada em C e D. Na A o dev manda o relatório inteiro por mensagem. Em B–D, a resposta é um recorte da tela. A própria *skill* já recomenda pedir de antemão o "*ask back*" para resultados longos. Nas repetições conferidas à mão, o Narrador só pediu o relatório completo depois que a resposta chegou cortada. [inferido] Isso explica parte dos pedidos extras, não todos: D2, com 5 repetições, é a rodada mais longa.

3. **A versão A não é a recomendação.** A A só funciona com Claude Code: exige MCP, `-append-system-prompt` e *hook*, exatamente o que a D01 recusou. O que ela ganha aqui é que o relatório chega inteiro. [inferido] Dá para levar isso para o protocolo agnóstico sem exigir cooperação: devolver no `ask` o texto inteiro do último turno, lido do histórico do emulador (*scrollback*) e não só da tela visível. O pedido de "*ask back*" também pode ir junto quando o pedido é grande. Essa é a etapa O5 do *backlog*, que já estava planejada. O critério de aceite agora é este benchmark: as repetições devem cair a zero sem aumentar os tokens.

4. **C contra D sem mecanismo identificado.** [medido] C e D têm a mesma *skill*, os mesmos papéis e a mesma detecção (1,6–2,3 s contra 1,6–1,9 s), e D processou mais tokens em todas as rodadas. [inferido] Candidatos:

   - a *statusline* de D muda o rodapé da tela que vira resposta;
   - a variação da estratégia do Narrador: na D1 ele montou um quadro de notas (*Analise*, *Contrato*, *Revisao*) e pediu revisão cruzada.

   Com três rodadas, não dá para separar uma coisa da outra.

5. **O papel de líder é parte do resultado.** Na A, o líder não tem Bash e lê o código com a ferramenta `Read`. Em B–D, o líder tem Bash, mas o usa quase só para a CLI, e lê menos código (de 0 a 9 leituras por rodada; nenhuma nas três rodadas da D). Nas quatro versões o líder delegou tudo e não editou nada, então essa diferença de permissão não mudou quem faz o trabalho.

6 Ameaças à validade

- **Amostra pequena**: três rodadas por versão. Nenhum teste passa de $p = 0{,}10$, e as diferenças entre versões vizinhas se sobrepõem. Os resultados são indicações, não provas.
- **Teto de qualidade**: 31/31 em todas as rodadas. A tarefa mede custo, não capacidade. Uma tarefa mais difícil pode inverter conclusões; um protocolo mais caro pode, por exemplo, pegar mais erros.
- **Uma tarefa, um fornecedor**: só Claude. O ganho principal de B–D (falar com Codex, Gemini e outros sem cooperação) não é medido aqui.
- **Ferramentas de hoje em versões antigas**: a A foi escrita para o Claude Code de julho e rodou no 2.1.300; o MCP e o *hook* ainda funcionam, mas o comportamento do agente mudou desde então.
- **Heurísticas**: "resposta cortada" é detectada por palavras e conferida à mão; a classificação das chamadas (coordenação, leitura, verificação) é por padrão de comando.
- ***Cache***: a métrica principal (tokens processados) não depende de acerto de *cache*, mas o custo em US$ depende. A ordem contrabalançada reduz o viés.
- **Ambiente**: Xvfb, um contêiner, um horário. O atraso de 25 s do Narrador e `IS_SANDBOX` são iguais para todas as versões.

7 Como reproduzir

Tudo está em `bench/orquestracao/` (skill `benchmark` do repositório):

```
git worktree add --detach $BENCH/vX <commit>  # uma por versão
$BENCH/build.sh A B C D                    # compila em $BENCH/bin/vX
Xvfb :77 -screen 0 1600x1000x24 &
cd bench/orquestracao && ./rodar_todas.sh         # 12 rodadas, ~50 min
python3 analisar.py $BENCH/runs/v?-r[1-9] && python3 agregar.py
cd docs/relatorios && pdflatex orquestracao-versoes.tex
```

Cada rodada guarda as transcrições, o placar, o *diff*, a lista de agentes e uma captura de tela em `$BENCH/runs/vX-rN/`.